主机发起一切. USB 是严格的主从结构，设备永远不能主动说话。即使是名为“interrupt”的传输类型，也只是“主机承诺每隔 `bInterval` 毫秒来问一次”：主机发 IN 令牌，设备有数据就回 DATA，没有就回 NAK。所谓“键盘中断”，其实是主机控制器替 CPU 轮询，问到数据后才用 IRQ 9 通知 CPU。

host channel 与 DMA. DWC2 有 8 个 host channel，每个 channel 是一组寄存器，描述“对哪个设备的哪个端点做一次什么传输”：`HCCHAR`（设备地址、端点号、方向、传输类型、最大包长、是否低速）、`HCSPLT`（split 事务参数）、`HCTSIZ`（字节数、包数、DATA0/1）、`HCDMA`（缓冲区的总线地址）、`HCINT`（完成、ACK、NAK、NYET、错误等状态）。本项目固定分工：ch0 做枚举时的 control 传输，ch6 给键盘的 interrupt-IN，ch4 给 MT7601U 的 bulk-IN。和 Arasan 不同，DWC2 自己用 DMA 读写内存，所以缓冲区在提交前要 clean、完成后要 invalidate 缓存，地址也要换成 `0xC0000000|PA` 的总线地址。

一切都在 Hub 后面. 树莓派 3 的 SoC 只有一个 USB 口，直接接板载 LAN9514，它内部是一个高速 Hub 加一块 USB 以太网卡（在 3B+ 上还有第二层 Hub）。外面插的键盘一般是低速（1.5 Mb/s）设备，而 DWC2 和 Hub 之间跑的是高速（480 Mb/s）。两种速率之间靠 Hub 里的事务转换器（TT）衔接，这就是后面 split transaction 的由来。

7.4.2 事务格式：Token、Data、Handshake 三相

上一小节说“主机发起一切”，但一次到底“发起”了什么？USB 总线上被独立调度的最小单位既不是字节也不是包，而是**事务**（transaction）。一次事务由主机发起，按固定顺序串起最多三种**包**（packet）：主机先发一个 **Token** 包，声明“要对哪个设备的哪个端点做一次什么操作”；接着是一个可选的 **Data** 包，方向取决于 Token 的类型；最后是一个 **Handshake** 包，由接收方确认或拒绝。规范里的图 8-30（bulk 事务）和图 8-36（control SETUP 事务）画的正是这套三相结构在不同传输类型下走过的分支。

每个包的场（field）布局是固定的：以 `SYNC` 开场让接收方对齐位边界，紧跟一个 `PID` 场标明包的类型，中间是地址/负载与校验，末尾以 `EOP`（end of packet）收束，如图 7.4。

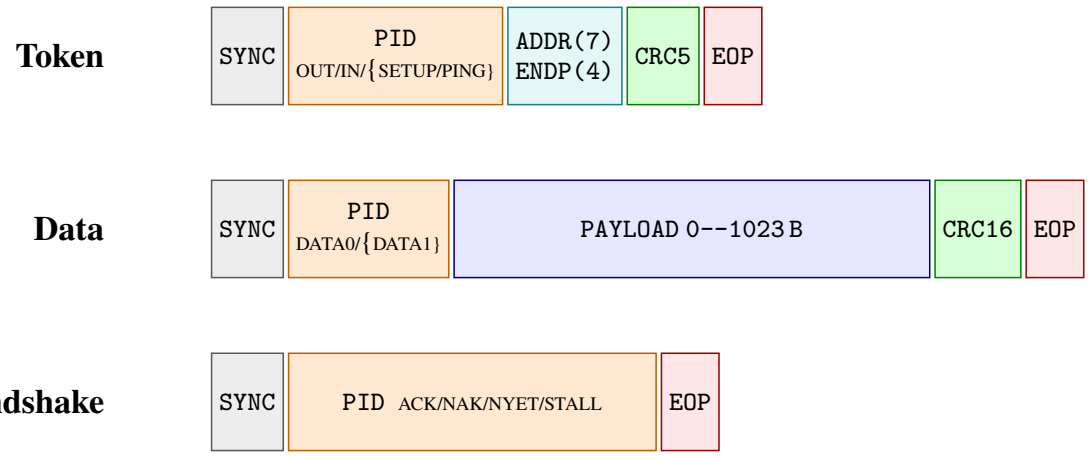


图 7.4: USB 三种包的场布局。Token 包用 CRC5 保护“地址 + 端点”，Data 包用 CRC16 保护负载，Handshake 包只有 SYNC/PID/EOP、既无负载也无 CRC 场。关键区别：**PID 场本身占 8 位（4 位本体再取反重发一次）**，双重编码已足以自校验，所以 PID 不需要额外 CRC，握手包也因此不必带 CRC

PID 决定了一个包在事务里扮演什么角色，表 7.2 按 Token/Data/Handshake 三类把本项目会遇到的 PID 列全。

三相结构在三种传输类型下走的路径略有不同，图 7.5 把控制 SETUP、IN、OUT 三条主线并排画出（框的颜色标出这个包由谁发出：灰框 = 主机、白框 = 设备）。

把这套三相格式对回本项目的三个 channel 就很清楚了：ch0 的枚举是反复做 control 事务——先 SETUP 三相把“读描述符/设配置”的请求送进去，再用若干 IN 或 OUT 事务搬运数据，最后用一个反方向的空 Data 事务做 STATUS；ch6 键盘每隔 `bInterval` 发一次 IN 事务，设备没按键就回 NAK、有按键才回 DATA1/DATA0；ch4 MT7601U 的 bulk-IN 同样发 IN，靠 DATA0/DATA1 的翻转做流控。下一小节的 split 事务，本质就是把这里的三